Universität Münster

Energy Trading 2026 - Web Development

Maurice Tarp

9th Apr 2026

University of Münster

living.knowledge

Outline

Introduction
- What is Web Development?
- Use Cases
- Examples

Areas of Web Development

Backend: Spring
- Introduction
- Example
- Annotations
- Alternatives

Frontend: Angular
- Components and Templating
- Signals
- Routing
- Alternatives

Introduction

Introduction

What is Web Development?

Introduction

What is Web Development?

- “The Web” has existed for a while now…
- but it went through several “iterations”: Web 1.0, Web 2.0, soon maybe Web 3.0?
- Web 1.0 is basically just static websites using HTML + CSS
- Web 2.0 marks the advent of more dynamic webpages and development of frameworks

https://paulgraham.com/web20.html

Introduction

Use Cases

- nowadays, web applications are far more than static webpages
- web applications are full-fledged applications, running in a web browser
- some applications are even shipped using various web technologies (or almost entire browsers! See `electron`) to ease development
- that means that web applications are highly portable by virtue of modern browsers being available for practically all consumer computing devices

Introduction

Examples

This is a web page.

There's not much here.

Just words.

And you're reading them.

We've become obsessed with fancy designs, responsive layouts, and scripts that do magical things.

But the most powerful tool on the web is still words.

I wrote these words, and you're reading them: *that's* magical.

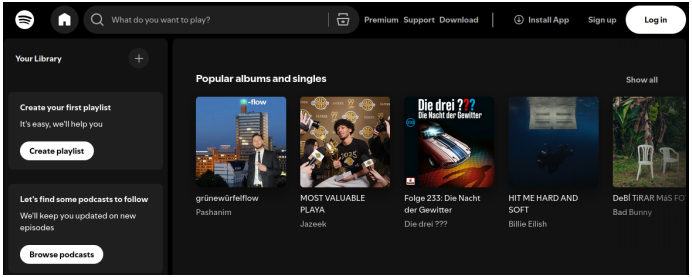
I'm in a little city in British Columbia; you're probably somewhere else. I wrote this early in the morning, June 20th, 2013; you're reading it at a different time. I wrote this on my laptop; you could be reading this on your phone, a tablet or a desktop.

You and I have been able to connect because *I wrote this* and *you're reading it.* **That's the web**. Despite our different locations, devices, and time-zones we can connect *here,* on a simple HTML page.

I wrote this in a text editor. It's 6 kB. I didn't need a Content Management System, a graphic designer, or a software developer. There's not much code on this page at all: just simple markup for paragraphs, hierarchy, and emphasis.

https://justinjackson.ca/words.html

- …so is this (an entire app even)



https://open.spotify.com/

Areas of Web Development

Areas of Web Development

Frontend

- basically: application UI
- dynamic websites, enabled by some framework
  - single-page applications! (SPA)
  - do no load/refresh entire pages, just what is needed on demand
- constant communication with the application backend that realizes functionality
- many frameworks support both client-side and server-side rendering of HTML

Areas of Web Development

Backend

- in many cases a server exposing an interface (oftentimes REST) for the frontend to communicate with
- handles application logic, communication with databases and responds to requests from the frontend
- does not need to be a single application!
- the backend can consist of multiple services, distributed over various systems
- redundancy, resilience and performance can be improved by spinning up multiple instances of services/backends (as long as the architecture allows for that)

Areas of Web Development

Persistence

- web applications use and/or provide data
- this data needs to be stored (persisted) somehow
- intuitively: database systems handle persistence of data
- the backend interacts with the database system using a persistence layer

Backend: Spring

Backend: Spring

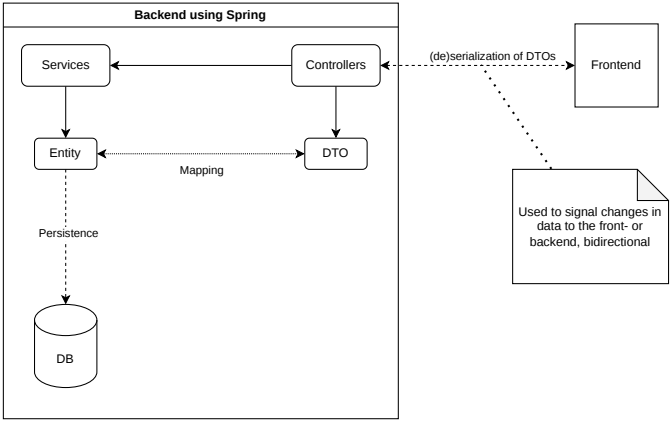
Introduction

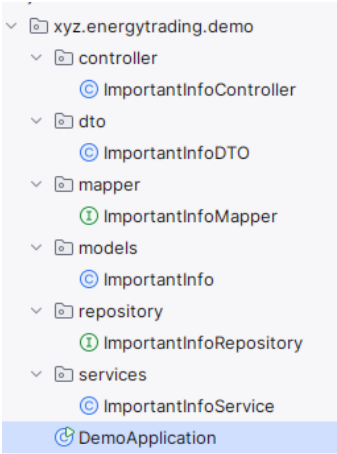
- modern Java web development framework (currently developed by VMware)
- advertises itself as a flexible, fast and productive development framework, with a strong community of Java developers
- comes with support for configurations and code generators to reduce boilerplate, convenience annotations as well as extensive support for various build systems and IDEs
- basic concepts: dependency injection, service-based application architecture, RESTful interfaces
- JPA is used for persistence

https://spring.io/

Backend: Spring

Example





Backend: Spring

Example

```
@Service
public class ImportantInfoService {
    @Autowired
    ImportantInfoRepository importantInfoRepository;
    @Autowired
    ImportantInfoMapper importantInfoMapper;

    public ImportantInfoService() {}  no usages

    public List<ImportantInfoDTO> getAllForName(String name) {  1 usage
        List<ImportantInfo> infosByName = importantInfoRepository.findAllByName(name)
                .orElse(Collections.emptyList());
        return importantInfoMapper.toDto(infosByName);
    }
```

Backend: Spring

Example

```java
@RestController
@RequestMapping("/important")
public class ImportantInfoController {

    @Autowired
    private ImportantInfoService importantInfoService;

    @GetMapping("/{name}")
    public ResponseEntity<List<ImportantInfoDTO>> getImportantInfoByName(@PathVariable String name) {
        return ResponseEntity.ok(importantInfoService.getAllForName(name));
    }
}
```

Backend: Spring

Example

```
@Entity
@Table(name = "important_information")
public class ImportantInfo {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(name = "name")  2 usages
    private String name;

    public Long getId() { return id; }
    public void setId(Long id) { this.id = id; }
    public String getName() { return name; }
    public void setName(String name) { this.name = name; }
}
```

Backend: Spring

Example

`http://localhost:8080/important/Test`

▶ returns:

JSON Raw Data Headers

Save Copy Collapse All Expanc

```
▾ 0:
    id:     1
    name:   "Test"
▾ 1:
    id:     2
    name:   "Test"
```

Backend: Spring

Annotations

- `@Service` and `@Autowired`
- `@RestController`, `@RequestMapping`, ...
- `@Entity`, `@Column`, `@Id`, ...

- Services and Dependency Injection
- RESTful interfaces and paths
- persistence and definition of data models

Backend: Spring

Alternatives

- for Java: Quarkus
- lots of frameworks out there, for various programming languages:
  - Ruby on Rails
  - Node.js
  - Python Django
  - and many more
- frameworks may take different approaches in what features and conveniences they provide

Frontend: Angular

Frontend: Angular

Components and Templating

- developed by Google
- runs using `Node.js`
- component-based approach
  - components describe parts of pages or even an entire page
  - used to encapsulate behavior, similar to how classes would
  - components utilize templates in the form of “enhanced” HTML + CSS styling
  - “enhanced” in the sense that parts of the component can be used within the HTML template, e.g. collections that are dynamically updated once they have changed
  - templates can even incorporate control flow like loops and branching
- the project provides learning resources: `https://angular.dev/tutorials`

Frontend: Angular

Signals

- signals realize another feature of Angular: *reactivity*
  - DOM is dynamically updated, actions are triggered, ...
  - all without explicitly implementing any to do so logic yourself!
- `signal` is a wrapper around a value, triggering different reactions once the underlying value changes
- this is how DOM updates and communication in between components and services can be achieved
- `computed` is a read-only signal, reacting to changes of other signals it observes

Frontend: Angular

Routing

- Angular also supports routing **within** the SPA
- with Angular, routing happens client-side after the first request has been made, enabling navigation
- routes are often defined in a central location, i.e. a `routes.js` file

```
export const routes: Routes = [
  {
    path: '',
    component: Root,
  },
  {
    path: 'page1',
    component: Page1,
  },
];
```

Frontend: Angular

Routing

- the current URL can also include information the application uses to determine i.e. how a request to the backend is made, apply filters, or just load required data
- for example, to get some data from a route

```
export const routes: Routes = [
  {
    path: 'page1/:someData',
    component: Page1,
  }
];
```

Frontend: Angular

Alternatives

- React, less of a framework and more of a library, used and developed by Meta
- Vue.js, adoption can happen incrementally
- Svelte, fairly new, but fast

Sources

- https://www.paulgraham.com/web20.html
- https://web.archive.org/web/20030810102320/http://devedge.netscape.com/viewsource/2003/inner-browsing/
- https://angular.dev/tutorials
- https://justinjackson.ca/words.html
- https://open.spotify.com/
- https://www.electronjs.org/

Thank you for your attention!

Questions?